Grand Canyon University
College of Science, Engineering and Technology

Neural Network Optimization: Tuning Strategies Across Architectures

Authors: Owen Lindsey & Tyler Friesen
Course: AIT-204 — Deep Learning
Instructor: Professor Artzi
Date: April 2026

A technical paper exploring the practical strategies, mathematical foundations, and architecture-specific considerations behind tuning neural networks, drawn from current research.

Abstract

Getting a neural network to actually work well turns out to be just as hard as designing one. In this paper, we dig into the research behind neural network optimization, specifically what makes the biggest difference in performance, how developers approach the tuning process, and whether there is a one-size-fits-all strategy or if architectures like CNNs, RNNs, and standard ANNs each need their own playbook. We pulled from several recent papers and surveys to piece together a practical picture of the current landscape. What we found is that while the broad strokes of tuning (get your learning rate right, regularize, don't skip data cleaning) apply everywhere, each architecture has its own quirks and failure modes that you have to account for. We include mathematical background and Python code throughout to keep things grounded.

Keywords: neural network optimization, hyperparameter tuning, learning rate scheduling, regularization, deep learning

Contents

1. Introduction

Anyone who has trained a neural network knows the feeling: you set up your model, hit train, and watch the loss curve do something completely unexpected. Maybe it flatlines. Maybe it spikes. Maybe it looks great on training data and falls apart on anything new. The gap between "I have a neural network" and "I have a neural network that actually works" is where tuning lives, and it is a bigger gap than most introductory courses let on.

The core problem is that training a deep network means searching a massive, non-convex loss landscape full of local minima, saddle points, and flat regions [1]. On top of that, the hyperparameter space is huge (learning rate, batch size, depth, width, dropout, optimizer, weight initialization) and these all interact with each other in ways that are hard to predict. Change the learning rate and suddenly your batch size choice matters differently. Add dropout and maybe you need a higher learning rate to compensate.

Different architectures make this even more complicated. A CNN processes spatial data with shared filters. An RNN carries hidden state across time steps. A basic feedforward ANN just maps inputs to outputs layer by layer. Each of these has its own set of things that tend to go wrong and its own bag of tricks for fixing them.

In this paper, we work through the following questions based on what we found in the research:

1. What has the biggest impact on how well a neural network performs?
2. What does a practical tuning process look like?
3. What should you realistically expect when you are done tuning?
4. Do CNNs, RNNs, and ANNs need fundamentally different approaches?
5. What are developers actually doing in practice right now?

2. Literature Review

2.1 Hyperparameter Optimization Surveys

Yu and Zhu [2] put together a solid survey of hyperparameter optimization methods, breaking them into grid search, random search, Bayesian optimization, and gradient-based methods. One of the more interesting takeaways is that random search consistently beats grid search when you have the same compute budget. This was originally shown by Bergstra and Bengio [3], and the reason makes intuitive sense: not all hyperparameters matter equally. Learning rate usually dominates everything else, and grid search wastes a lot of evaluations testing fine-grained differences in parameters that barely matter.

2.2 Learning Rate and Optimization Algorithms

Pretty much every source we looked at agreed on one thing: the learning rate is the single most important hyperparameter [1]. Smith [4] takes this further with the idea of "super-convergence," where training with surprisingly large learning rates using a cyclical schedule, and getting both faster convergence and better final performance. The learning rate range test he describes (sweep from tiny to huge, plot the loss, pick a value just below the minimum) has become a go-to technique.

On the optimizer side, adaptive methods like Adam [5] have become the default starting point because they are less sensitive to the initial learning rate. But there is an interesting counterpoint from Wilson et al. [6], who show that well-tuned SGD with momentum often matches or beats Adam on test accuracy. The catch is "well-tuned," since SGD needs more careful scheduling to get there.

2.3 Regularization and Generalization

Overfitting is the constant enemy, and the research covers several ways to fight it. Dropout [7] randomly shuts off neurons during training, which works like training a bunch of smaller networks and averaging them. Batch normalization [8] normalizes layer inputs and has the nice side effect of smoothing the loss landscape, which lets you use higher learning rates. Luo et al. [9] found that adjusting regularization strength during training can outperform keeping it fixed, which makes sense because the model's needs change as it learns.

2.4 Architecture-Specific Optimization

This is where things diverge. For CNNs, transfer learning and data augmentation are basically mandatory [11]. For RNNs, you need gradient clipping and careful weight initialization because the repeated matrix multiplications across time steps make gradients either explode or vanish [12]. Transformers need warmup schedules and careful attention to layer normalization [13]. The takeaway from the literature is that there is no escaping architecture-specific knowledge.

2.5 Automated Machine Learning

The complexity of all this has pushed people toward automation. Neural Architecture Search can find architectures that compete with hand-designed ones [14], and frameworks like Optuna make Bayesian hyperparameter search much more accessible. Feurer and Hutter [15] point out that combining algorithm selection with hyperparameter optimization (what they call CASH) often does better than tuning any single algorithm in isolation.

3. What Impacts the Performance of a Neural Network?

Not everything matters equally. Based on the research, we can roughly rank the factors that affect performance.

3.1 Data Quality and Quantity

This comes first for a reason. We have seen this firsthand in our own class projects, and no amount of architecture tweaking or hyperparameter tuning saves a model trained on messy data.

Deep networks are hungry for data, and performance tends to follow a power law with dataset size: error $\propto n^{-\alpha}$, where α depends on the problem [16]. Label noise is especially damaging; even 5–10% mislabeled examples can drop accuracy by several points.

Preprocessing matters too. Normalizing inputs to zero mean and unit variance is one of those things that seems simple but makes a real difference:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}, \quad \mu = \frac{1}{N}\sum_{j=1}^{N} x_j, \quad \sigma = \sqrt{\frac{1}{N}\sum_{j=1}^{N}(x_j - \mu)^2} \tag{1}$$

Without this, features on different scales pull the gradient in uneven directions and convergence slows way down. Here is how it looks in PyTorch:

Listing 1: Data normalization in PyTorch

```
import torch

# Compute stats from training set only
train_mean = X_train.mean(dim=0)
train_std  = X_train.std(dim=0)

# Apply to both splits (avoids data leakage)
X_train_norm = (X_train - train_mean) / (train_std + 1e-8)
X_test_norm  = (X_test - train_mean) / (train_std + 1e-8)
```

3.2 Architecture Design

Choosing the right architecture is really about matching the model's inductive biases to the structure of your data. A few things consistently matter:

Depth vs. width. Deeper networks can represent more complex functions, but they are harder to train. The universal approximation theorem tells us a wide enough single-layer network can approximate anything, but in practice, going deeper gets you the same representational power with far fewer parameters.

Capacity. Too few parameters and the model underfits. Too many and it memorizes training noise. Finding the sweet spot is a recurring theme in tuning.

Skip connections. Residual connections [10] were a breakthrough for training deep networks. The idea is straightforward: let the gradient bypass layers through an identity shortcut:

$$\mathbf{y} = \mathscr{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{2}$$

This keeps gradients from vanishing through very deep stacks.

3.3 Hyperparameter Selection

These are the knobs you turn during tuning. Table 1 ranks the key ones by how much they typically affect results.

Table 1: Key hyperparameters ranked by typical impact

Hyperparameter	Impact	What it does
Learning rate	Critical	Step size in gradient descent
Batch size	High	Gradient noise vs. speed
Network depth	High	Representational capacity
Hidden units	Medium	Per-layer capacity
Dropout rate	Medium	Regularization (0.1–0.5)
Weight decay	Medium	L2 penalty on weights
Optimizer	Medium	Adam vs. SGD vs. others

3.4 Optimization Algorithm

The optimizer determines how the network moves through the loss landscape. Standard SGD takes a step proportional to the gradient:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta \mathscr{L}(\theta_t) \tag{3}$$

Adam [5] gets fancier by keeping running averages of the gradient and its square, giving each parameter its own effective learning rate:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2 \tag{5}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \tag{6}$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected versions. This is especially helpful when your features live on very different scales.

4. How Should One Approach Tuning a Network?

4.1 A Practical Tuning Workflow

After going through the literature, we think the most useful way to present this is as a step-by-step process. It is tempting to just start tweaking things, but a structured approach saves a lot of wasted time.

Step 1: Establish a baseline. Start with something simple and known to work. Use default hyperparameters. The goal here is just to get a model that trains without crashing and does better than random.

Step 2: Verify the pipeline. This one is easy to skip and painful when you do. Feed the model a tiny batch (32 samples) and train until it memorizes them. If the loss does not go to near-zero, you have a bug, not a tuning problem.

Listing 2: Sanity check: can the model memorize a small batch?

```
model = SimpleNet(input_dim=10, hidden_dim=128, output_dim=1)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

small_X, small_y = X_train[:32], y_train[:32]
for epoch in range(200):
    loss = criterion(model(small_X), small_y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
# If loss doesn't approach 0, something is broken
```

Step 3: Find the right learning rate. Smith's learning rate range test [4] is the go-to here. Sweep the LR from very small (1e-7) to very large (10) over a few hundred iterations, and plot loss vs. LR. Pick a value about one order of magnitude below where loss is lowest.

Step 4: Regularize. Watch the gap between training and validation loss. When training loss keeps dropping but validation loss stalls or rises, you are overfitting. Add dropout, weight decay, or data augmentation rather than shrinking the model.

Step 5: Scale up. Gradually increase model capacity (more layers, more units). Keep monitoring that train/val gap. Use a learning rate schedule, as the one-cycle policy in particular has been shown to enable much faster convergence [4].

Listing 3: Learning rate range test

```python
from torch.optim.lr_scheduler import ExponentialLR
lr_min, lr_max, steps = 1e-7, 10, 300
gamma = (lr_max / lr_min) ** (1 / steps)
opt = optim.SGD(model.parameters(), lr=lr_min)
sched = ExponentialLR(opt, gamma=gamma)
lrs, losses = [], []
for step in range(steps):
    X_b, y_b = next(iter(train_loader))
    loss = criterion(model(X_b), y_b)
    opt.zero_grad(); loss.backward(); opt.step(); sched.step()
    lrs.append(opt.param_groups[0]['lr'])
    losses.append(loss.item())
# Plot lrs vs losses -- pick lr just below the dip
```

4.2 Learning Rate Schedules

Almost nobody trains with a fixed learning rate anymore. The most common schedules we found in the literature:

Step decay drops the LR by a factor every few epochs: $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$

Cosine annealing smoothly ramps it down:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos(\tfrac{t}{T}\pi)\right) \tag{7}$$

One-cycle ramps up and then back down, and is behind the "super-convergence" results Smith reported:

Listing 4: One-cycle policy in PyTorch

```python
from torch.optim.lr_scheduler import OneCycleLR

optimizer = optim.SGD(model.parameters(), lr=0.01,
                      momentum=0.9, weight_decay=1e-4)
scheduler = OneCycleLR(
    optimizer, max_lr=0.1, epochs=100,
    steps_per_epoch=len(train_loader),
    pct_start=0.3, anneal_strategy='cos')

for epoch in range(100):
    for X_b, y_b in train_loader:
        loss = criterion(model(X_b), y_b)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        scheduler.step()  # call per batch, not per epoch
```

4.3 Early Stopping

Early stopping is the simplest form of regularization: track validation loss, and stop training when it has not improved for p consecutive epochs (the "patience"). Save the best checkpoint and use that as your final model.

5. What Can Be Expected at the End of Tuning?

It is worth setting realistic expectations. Tuning is not going to hand you a perfect model.

Diminishing returns. The first round of serious tuning (getting the learning rate right, choosing an appropriate architecture, cleaning the data) captures the bulk of the gains. Everything after that is incremental. We saw this pattern repeatedly in the literature, and it matches our experience in class projects.

Trade-offs everywhere. Accuracy, speed, and memory are in tension. A bigger model might be more accurate but slower to train and deploy. Tuning explores this trade-off space, but it does not eliminate it.

Results vary between runs. Random initialization, data shuffling, and GPU non-determinism mean that running the same hyperparameters twice can give different results. The responsible thing is to run multiple seeds and report mean $\pm$ standard deviation.

Every architecture has a ceiling. No amount of tuning will make a feedforward MLP beat a CNN on image classification. An RNN will always struggle with extremely long sequences compared to a Transformer. Tuning gets you closer to the ceiling, but the ceiling is set by the architecture.

6. Generic vs. Architecture-Specific Tuning

The workflow from Section 4.1 applies broadly. But once you get past the basics, each architecture has its own priorities and its own ways of breaking.

6.1 ANNs / MLPs

Feedforward networks are the simplest starting point. For tabular data, they are still competitive with more exotic architectures. The main tuning considerations:

- **Width over depth.** For tabular tasks, 2–4 layers with plenty of hidden units tends to work better than going very deep. Research suggests that for structured/tabular data, the benefit of adding layers drops off sharply after about 4, while increasing width continues to help.
- **He initialization** keeps the variance of activations stable across layers: $W \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$. Without proper initialization, activations either explode or vanish as they propagate through the network, making training slow or impossible.

- **Dropout** in the 0.1–0.5 range is the primary regularizer. Higher values work better for larger, more overfit-prone networks; lower values are safer when you are not sure.
- **Batch normalization** after each linear layer stabilizes training and lets you push the learning rate higher. The pattern is Linear → BatchNorm → ReLU → Dropout.
- **Main failure mode:** overfitting on small datasets. MLPs have no built-in inductive bias like spatial locality (CNNs) or temporal structure (RNNs), so they rely entirely on the data to learn useful representations.

The listing below shows what a well-tuned feedforward network looks like in practice, pulling together He initialization, batch normalization, and dropout in a clean pattern:

Listing 5: A well-structured feedforward network

```
class TunedANN(nn.Module):
    def __init__(self, in_dim, hidden=256, out=10, drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden),
            nn.BatchNorm1d(hidden), nn.ReLU(), nn.Dropout(drop),
            nn.Linear(hidden, hidden),
            nn.BatchNorm1d(hidden), nn.ReLU(), nn.Dropout(drop),
            nn.Linear(hidden, out))
        for m in self.modules():                          # He init
            if isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight)
    def forward(self, x): return self.net(x)
```

6.2 CNNs

CNNs are where transfer learning really shines. For most vision tasks, starting from a pre-trained model (ResNet, EfficientNet, etc.) and fine-tuning is drastically better than training from scratch [11]. The typical approach:

1. Freeze all layers, replace the classification head, train only the new layers with a moderate LR (∼1e-3).
2. Unfreeze later layers and fine-tune with a smaller LR (∼1e-4 to 1e-5).

Listing 6: Transfer learning with progressive unfreezing

```
model = models.resnet18(weights='IMAGENET1K_V1')

for p in model.parameters():       # freeze everything
    p.requires_grad = False

model.fc = nn.Sequential(          # new head
    nn.Linear(512, 256), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, num_classes))

# Phase 1: train head only
opt = optim.Adam(model.fc.parameters(), lr=1e-3)

# Phase 2: unfreeze last conv block, lower lr
for p in model.layer4.parameters():
    p.requires_grad = True
opt = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}])
```

Data augmentation (random crops, flips, color jitter) is also essential for CNNs because it effectively multiplies your dataset and is one of the cheapest forms of regularization.

6.3 RNNs / LSTMs / GRUs

Recurrent networks have unique challenges. The repeated matrix multiplications across time steps make gradients either vanish (for long sequences) or explode. Two things are non-negotiable:

Gradient clipping rescales the gradient when its norm exceeds a threshold τ:

$$\hat{g} = \begin{cases} g & \text{if } \|g\| \leq \tau \\ \frac{\tau}{\|g\|} g & \text{if } \|g\| > \tau \end{cases} \tag{8}$$

We used this in our own LSTM temperature forecasting project earlier this semester and it made a noticeable difference. Without clipping, training was erratic.

Use LSTMs or GRUs instead of vanilla RNNs for anything longer than about 20–30 time steps. The gating mechanisms are specifically designed to let gradients flow over longer distances.

Listing 7: LSTM training with gradient clipping

```
class TunedLSTM(nn.Module):
    def __init__(self, in_dim, hidden=128, layers=2, drop=0.2):
        super().__init__()
        self.lstm = nn.LSTM(in_dim, hidden, layers,
                            batch_first=True,
                            dropout=drop if layers > 1 else 0)
        self.fc = nn.Linear(hidden, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

model = TunedLSTM(in_dim=1)
opt = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(num_epochs):
    for X_b, y_b in train_loader:
        loss = criterion(model(X_b), y_b)
        opt.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
```

6.4 Comparing the Three

Table 2 puts the key differences side by side.

Table 2: How tuning priorities differ across architectures

Aspect	ANN / MLP	CNN	RNN / LSTM
Top priority	Width, dropout	Transfer learning	Gradient clipping
Regularization	Dropout, wt. decay	Data augmentation	Recurrent dropout
Learning rate	1e-3 to 1e-2	1e-5 (pretrained)	1e-3 with warmup
Initialization	He / Xavier	Pre-trained weights	Orthogonal
Failure mode	Overfitting	Poor augmentation	Exploding gradients

7. Key Practices in Use Today

7.1 Automated Hyperparameter Search

Manually trying different hyperparameter combinations does not scale. Bayesian optimization models the relationship between hyperparameters and performance as a Gaussian process, then

intelligently picks the next configuration to try:

$$\lambda^{*} = \arg\max_{\lambda} \; \alpha(\lambda \mid \mathcal{D}_{1:t}) \tag{9}$$

where α is an acquisition function like Expected Improvement. In practice, Optuna makes this surprisingly easy:

Listing 8: Hyperparameter search with Optuna

```python
import optuna

def objective(trial):
    lr   = trial.suggest_float('lr', 1e-5, 1e-1, log=True)
    h    = trial.suggest_int('hidden', 64, 512, step=64)
    drop = trial.suggest_float('dropout', 0.0, 0.5)

    model = build_model(h, drop)
    opt   = optim.Adam(model.parameters(), lr=lr)
    return train_and_evaluate(model, opt)

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=100)
print(f"Best: {study.best_params}")
```

7.2 Mixed-Precision Training

Modern GPUs can do math in 16-bit floating point about twice as fast as 32-bit, and PyTorch's AMP (Automatic Mixed Precision) makes it easy to take advantage of this:

Listing 9: Mixed-precision training

```python
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()
for X_b, y_b in train_loader:
    opt.zero_grad()
    with autocast():
        loss = criterion(model(X_b), y_b)
    scaler.scale(loss).backward()
    scaler.step(opt)
    scaler.update()
```

7.3 Experiment Tracking and Reproducibility

Keeping track of what you tried and what worked is essential once you go beyond a handful of experiments. Tools like Weights & Biases and MLflow log hyperparameters, metrics, and model checkpoints automatically. We also found that seeding everything (Python, NumPy, PyTorch,

CUDA) and reporting results over multiple runs is considered standard practice in the research community.

7.4 Ensembles

Averaging predictions from 3–5 independently trained models ($\hat{y} = \frac{1}{K}\sum_{k=1}^{K} f_k(\mathbf{x})$) is one of the most reliable ways to squeeze out an extra 1–3% accuracy. It is simple, it works, and it is used in almost every competitive ML setting.

8. Mathematical Background Summary

To tie the math together in one place: the core problem is minimizing the regularized empirical loss:

$$\theta^* = \arg\min_{\theta} \frac{1}{N}\sum_{i=1}^{N} \mathscr{L}\big(f(\mathbf{x}_i;\theta), y_i\big) + \lambda\,\Omega(\theta) \tag{10}$$

The regularization term $\Omega(\theta)$ is usually L2 (weight decay: $\frac{1}{2}\|\theta\|_2^2$) or L1 (sparsity: $\|\theta\|_1$). For the loss $\mathscr{L}$, classification tasks typically use cross-entropy ($-\sum_c y_c \log \hat{y}_c$) and regression tasks use mean squared error ($\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$).

Everything in tuning ultimately connects back to the bias-variance trade-off:

$$\mathbb{E}[\text{error}] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \tag{11}$$

Making the model more complex reduces bias but increases variance. Regularization does the opposite. The tuning process is, at its core, about finding the sweet spot between these two forces for your particular problem.

9. Conclusion

After going through the research, a few things stand out.

Data quality and learning rate matter more than everything else. Before spending hours on architecture search or fancy hyperparameter sweeps, clean your data and run a learning rate range test. These two steps alone will get you most of the way there.

There is a general tuning workflow that applies across architectures: start simple, verify the pipeline, find the right learning rate, add regularization as needed, then scale up. But the details differ significantly. CNNs live and die by transfer learning and data augmentation. RNNs need gradient clipping to stay stable. ANNs on tabular data need careful control over capacity and dropout.

Diminishing returns are real. That first round of disciplined tuning captures 80–90% of the achievable improvement, and everything after that is incremental. At some point, it is more productive to improve your data than to keep tweaking hyperparameters.

The field is also moving toward automation. Tools like Optuna for hyperparameter search, mixed-precision training, and experiment tracking platforms are becoming standard. Knowing how to use them is as important as understanding the math behind gradient descent.

If there is one thing we would tell someone just getting started with tuning: do not skip the fundamentals. Understand what each hyperparameter actually does, verify your pipeline before you optimize, and let the data guide your decisions.

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[2] T. Yu and H. Zhu, "Hyper-Parameter Optimization: A Review of Algorithms and Applications," *arXiv preprint arXiv:2003.05689*, 2020.

[3] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[4] L. N. Smith, "A Disciplined Approach to Neural Network Hyper-Parameters: Part 1," *arXiv preprint arXiv:1803.09820*, 2018.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015.

[6] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, "The Marginal Value of Adaptive Gradient Methods in Machine Learning," in *Advances in NeurIPS*, 2017.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *JMLR*, vol. 15, pp. 1929–1958, 2014.

[8] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. ICML*, 2015.

[9] P. Luo, X. Wang, W. Shao, and Z. Peng, "Towards Understanding Regularization in Batch Normalization," in *Proc. ICLR*, 2019.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. CVPR*, pp. 770–778, 2016.

[11] T. He *et al.*, "Bag of Tricks for Image Classification with CNNs," in *Proc. CVPR*, 2019.

[12] R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proc. ICML*, 2013.

[13] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in NeurIPS*, 2017.

[14] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning Transferable Architectures for Scalable Image Recognition," in *Proc. CVPR*, 2018.

[15] M. Feurer and F. Hutter, "Hyperparameter Optimization," in *Automated Machine Learning*, Springer, pp. 3–33, 2019.

[16] J. Hestness *et al.*, "Deep Learning Scaling is Predictable, Empirically," *arXiv preprint arXiv:1712.00409*, 2017.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers," in *Proc. ICCV*, 2015.